

OFFICIAL MENTORSHIP TEXTBOOK

OOPS! .

From Code to Professional Game Developer

Author & Student: Khalid Abdullah

Course Engine: Phaser 3.80.1 • Web Audio API • 2.5D Architecture

Curriculum: Game Developer Mentor Subagent

Baseline Milestone: Volume 1 (v1.03 Baseline) • August 2026

Table of Contents — Complete Curriculum

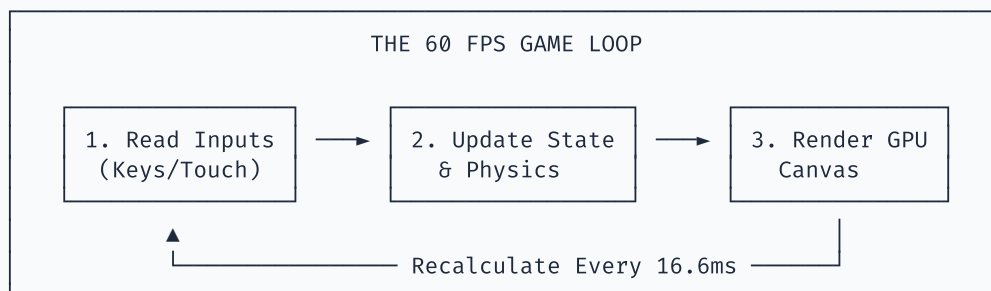
Part	Chapter	Title	Status
Part I	Chapter 01	The Game Loop, Phaser 3 Architecture & Single Frame Anatomy	Complete (Included)
Part I	Chapter 02	JavaScript Foundations for Game Developers	In Progress
Part I	Chapter 03	Git, Release Checkpoints & Versioning for Games	Planned
Part II	Chapter 04	Phaser 3 Core Architecture & Scale Manager	Planned
Part II	Chapter 05	Scene Lifecycle Pipeline: init, preload, create, update	Planned
Part II	Chapter 06	Player Systems & Procedural Sprite Generation	Planned
Part II	Chapter 07	Arcade Physics: Gravity, Velocity & Colliders vs Overlaps	Planned
Part II	Chapter 08	Multi-Touch Gamepad & Keyboard Input Engineering	Planned
Part III	Chapter 09	Camera Viewports, Boundaries & Parallax Math	Planned
Part III	Chapter 10	Data-Driven Level Design: World 1 Architecture (30 Stages)	Planned
Part III	Chapter 11	Troll Puzzle Mechanics: Vanishing Floors & Flee Doors	Planned
Part III	Chapter 12	2.5D Stylized Rendering: Bevels, Facets & Desert Parallax	Planned
Part IV	Chapter 13	Procedural Audio Synthesis with the Web Audio API	Planned
Part IV	Chapter 14	Mobile & Responsive Engineering: Dual-Orientation	Planned
Part IV	Chapter 15	Professional Debugging: Chrome CDP & Regression Testing	Planned
Part V	Chapter 17	Monetization: Google H5 Games Ads & 7-Death Reward State Flow	Planned
Part V	Chapter 19	Modern Web Game SEO: Sitemaps, Robots & Schema.org	Planned

Mentorship Philosophy: Never treat code as a black box. The goal of this textbook is not merely to build OOPS!, but to give you the mental models, vocabulary, and hands-on skills to create your own original games from scratch without AI assistance.

Chapter 01: The Game Loop, Phaser 3 & Single Frame Anatomy

1. Concept: The Optical Illusion of Games

Every video game is fundamentally an infinite mathematical loop executing 60 times per second (60 FPS). In each 16.6ms slice of time, the game loop processes three sequential steps:



2. The 3 Core Lifecycle Methods in Phaser 3

- **init(data):** Runs once when a scene starts. Receives parameters (e.g. current level index or death count) and initializes variables.
- **create():** Runs once after textures exist. Spawns player sprites, builds static platform geometry, sets up physics colliders, and registers keyboard listeners.
- **update(time, delta):** Runs continuously 60 times every second. Checks continuous keypresses (is the right arrow held down?), updates sprite velocities, and shifts background parallax.

3. OOPS! Engine Configuration Breakdown

In `game.js`, Phaser is initialized with this master configuration:

```
var VW = 960;
var VH = 540;

var config = {
  type: Phaser.AUTO,
  parent: "game-container",
  width: VW,
  height: VH,
  backgroundColor: "#0a0000",
  scale: {
    mode: Phaser.Scale.FIT,
    autoCenter: Phaser.Scale.CENTER_BOTH
  },
  physics: {
    default: "arcade",
    arcade: {
      gravity: { y: 1550 },
      debug: false
    }
  },
  scene: [ BootScene, IntroScene, WorldSelectScene, GameScene, WorldCompleteScene ]
};

var game = new Phaser.Game(config);
```

Property	What It Does in OOPS!	Why It Matters to Game Developers
<code>type: Phaser.AUTO</code>	Selects WebGL on modern GPUs; falls back to Canvas 2D on older devices.	Ensures maximum performance with 100% hardware compatibility.
<code>width: 960, height: 540</code>	Virtual logical coordinates (16:9 aspect ratio).	You program physics once for 960×540; it scales cleanly to any phone, tablet, or 4K monitor.
<code>scale.mode: Scale.FIT</code>	Scales canvas while strictly preserving the 16:9 ratio.	Prevents sprite stretching and distortion on weird aspect ratio screens.
<code>gravity: { y: 1550 }</code>	Arcade Physics pulls downward with 1550 px/s ² acceleration.	Produces snappy, responsive platformer jumps without floaty moon physics.

Common Beginner Mistake: Never create game objects inside `update()` ! Because `update()` runs 60 times every second, creating a sprite inside it will spawn 600 sprites in 10 seconds, quickly crashing your browser memory.

4. Diagnostic Quiz & Knowledge Check

1. Why does OOPS! use a fixed virtual resolution of 960×540 rather than setting canvas width to `window.innerWidth` ?
2. What is the difference between `this.physics.add.collider` and `this.physics.add.overlap` ?
3. In Phaser 3 coordinates, if an object's `y` value increases, does it move upwards or downwards?
4. What happens if you move a player using `this.player.x += 5` instead of `setVelocityX(200)` ?